

Deep Research: Avatrada 57 Topic 033

Engineering Report: Wash Sale Prevention and Capital Rotation Logic

Report ID: ER-WS-2023-09-14 **Author:** Autonomous Technical Researcher
Subject: Deep-Dive Analysis of the Wash Sale "Capital Rotation" System Component

1.0 Executive Summary

This report provides a rigorous engineering analysis of the "Wash Sale 'Capital Rotation' Logic" specified in the source document. The core challenge is to programmatically enforce the IRS wash sale rule—specifically, preventing re-entry into a security sold at a loss for 30 days—while simultaneously enabling a "capital rotation" strategy into highly correlated but not "substantially identical" securities.

The analysis concludes that a multi-factor, risk-based approach is necessary to programmatically assess "substantially identical" status. A simple symbol block is insufficient to be fully compliant, and a purely correlation-based approach is too broad. We propose a system architecture that combines a **Restricted Securities Ledger** with a **Pre-Trade Compliance Gateway** that queries a **Substantially Identical Securities (SIS) Analysis Module**. This module uses a weighted model of security metadata (CUSIP, Issuer, Index), quantitative analysis (correlation), and holdings data (ETF composition overlap) to generate a compliance decision.

2.0 Technical Deconstruction

The system can be broken down into four primary logical components:

2.1 Loss Realization Event Trigger

This is the entry point for the logic. It activates upon the execution of a `SELL` order that results in a capital loss.

- **Mechanism:** The system must have access to the position's cost basis. Upon a `SELL` execution, it calculates the realized Profit/Loss (P/L).
- **Formula:** $\text{Realized P/L} = (\text{Sell Price} - \text{Average Cost Basis}) * \text{Quantity} - \text{Commissions}$
- **Trigger Condition:** If $\text{Realized P/L} < 0$, the wash sale monitoring logic is initiated for the specific symbol (e.g., `SPY`).

2.2 Restricted Securities Ledger

This component acts as the system's stateful memory of wash sale restrictions. It is a persistent data store.

- **Architecture:** A key-value store (like Redis) is ideal for its performance and built-in support for time-to-live (TTL) keys. A relational database (like PostgreSQL) can also be used for greater auditability.
- **Data Schema:** Each entry in the ledger must contain:
 - `user_id`: The account the restriction applies to.
 - `restricted_symbol`: The CUSIP or ticker of the security sold at a loss (e.g., `SPY`).
 - `loss_date`: The timestamp of the sale that triggered the restriction.
 - `restriction_expiry`: The timestamp when the restriction lifts (`loss_date` + 31 days to be conservative).
 - `original_trade_id`: A reference to the sell order for auditing.

2.3 Pre-Trade Compliance Check

This is a synchronous, blocking gateway that intercepts all `BUY` orders before they are sent to the execution venue.

- **Mechanism:** For any incoming `BUY` order, this service queries the **Restricted Securities Ledger** and the **SIS Analysis Module**.

- **Logic Flow:**

1. Receive `BUY` request for `Symbol X`.
2. Query **Restricted Securities Ledger** for active restrictions on `Symbol X`. If found, **REJECT** the trade.
3. Query **SIS Analysis Module**: "Is `Symbol X` substantially identical to any other actively restricted symbol for this user?"
4. If the SIS module returns `true`, **REJECT** the trade.
5. If both checks pass, **APPROVE** the trade and pass it to the order router.

2.4 Substantially Identical Securities (SIS) Analysis Module

This is the intellectual core of the system, addressing the "Deep Research Prompt." It determines if two securities are "substantially identical" based on IRS guidelines. Since the IRS definition is intentionally ambiguous, a programmatic solution must be based on a defensible, multi-factor model.

- **Factor 1: Unique Identifiers (CUSIP/ISIN)**

- **Mechanism:** The most definitive test. If two securities share the same CUSIP, they are identical. This is the baseline check.
- **Example:** A common stock is always identical to itself.

- **Factor 2: Security Metadata**

- **Mechanism:** Compare fundamental metadata. For ETFs like SPY, VOO, and IVV:
 - **Issuer:** State Street (SPY) vs. Vanguard (VOO) vs. BlackRock (IVV). Different issuers are a very strong signal that the securities are *not* substantially identical. This is the primary justification used by financial advisors for tax-loss harvesting.
 - **Underlying Index:** All three track the S&P 500. This indicates high correlation but does not make them identical.

- **Expense Ratio:** SPY (0.09%), VOO (0.03%), IVV (0.03%). The difference in fees leads to different performance over time, further supporting the "not identical" argument.

- **Factor 3: Holdings Overlap Analysis**

- **Mechanism:** For funds (ETFs, mutual funds), perform a "look-through" analysis of their underlying holdings. This provides a quantitative measure of similarity.
- **Formula (Weighted Portfolio Overlap):** `` Let A and B be two ETFs. Let $w_A(i)$ be the weight of holding i in ETF A. Let $w_B(i)$ be the weight of holding i in ETF B.

$$\text{Overlap} = \sum_i \min(w_A(i), w_B(i)) \text{ for all } i \text{ in } (\text{Holdings}_A \cap \text{Holdings}_B)$$
 `` * **Interpretation:** An overlap score of >98% might be flagged as high risk, while an overlap of 95% might be considered acceptable. SPY, VOO, and IVV would have a very high overlap score, but combined with Factor 2 (different issuers/structure), they are generally considered distinct.

3.0 Implementation Strategy

3.1 Data Models and Pseudocode

SQL Schema for Restricted Securities Ledger:

```
CREATE TABLE restricted_securities (  
  id SERIAL PRIMARY KEY,  
  user_id VARCHAR(255) NOT NULL,  
  restricted_cusip VARCHAR(9) NOT NULL,  
  restricted_symbol VARCHAR(10),  
  loss_date TIMESTAMP WITH TIME ZONE NOT NULL,  
  restriction_expiry TIMESTAMP WITH TIME ZONE NOT NULL,  
  triggering_trade_id VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_user_expiry ON restricted_securities (user_id,
restriction_expiry);
```

Core Logic (Python Pseudocode):

```
import datetime

# Assume a database connection object 'db' and a 'sis_module'
# sis_module.are_substantially_identical(cusip1, cusip2) -> bool

def on_position_close(event):
    """Triggered after a sell order is filled."""
    if event.realized_pl < 0:
        expiry_date = event.trade_date + datetime.timedelta(days=31)
        db.execute(
            "INSERT INTO restricted_securities (user_id, restricted_cusip,
loss_date, restriction_expiry) VALUES (%s, %s, %s, %s)",
            (event.user_id, event.cusip, event.trade_date, expiry_date)
        )

def is_trade_allowed(user_id, proposed_cusip):
    """Pre-trade compliance check."""
    # 1. Check for direct restriction on the proposed CUSIP
    direct_restriction = db.query(
        "SELECT 1 FROM restricted_securities WHERE user_id = %s AND
restricted_cusip = %s AND restriction_expiry > NOW()",
        (user_id, proposed_cusip)
    )
    if direct_restriction:
        print(f"REJECT: Direct restriction on {proposed_cusip}.")
        return False

    # 2. Check for restrictions on substantially identical securities
    active_restrictions = db.query(
        "SELECT restricted_cusip FROM restricted_securities WHERE user_id = %s
AND restriction_expiry > NOW()",
        (user_id,)
    )
```

```

    for restricted_cusip in active_restrictions:
        if sis_module.are_substantially_identical(proposed_cusip,
restricted_cusip):
            print(f"REJECT: {proposed_cusip} is substantially identical to
restricted {restricted_cusip}.")
            return False

# 3. If all checks pass
print("APPROVE: Trade is compliant.")
return True

```

3.2 Building the SIS Analysis Module

This module requires access to a security master database and potentially a data provider for ETF holdings.

1. **Establish a "Known Pairs" Cache:** Manually define and cache relationships for common capital rotation pairs. This is the most direct way to implement the specification. `json { "SPY_CUSIP": { "allow_rotation_to": ["V00_CUSIP", "IVV_CUSIP"], "substantially_identical": ["SPY_OPTIONS_CUSIPS"] } }`
2. **Develop a Heuristic Scoring Function:** For unknown pairs, use a rules-based engine. `python def are_substantially_identical(cusip1, cusip2): sec1 = security_master.get_details(cusip1) sec2 = security_master.get_details(cusip2)`

```

# Rule 1: Identical CUSIP
if cusip1 == cusip2: return True

# Rule 2: Common stock and options on that stock
if sec1.type == 'STOCK' and sec2.type == 'OPTION' and sec1.symbol ==
sec2.underlying_symbol:
    return True
# (add inverse check)

# Rule 3: ETFs (the core problem)
if sec1.type == 'ETF' and sec2.type == 'ETF':
    # Different issuers is a strong signal of being distinct

```

```

    if sec1.issuer != sec2.issuer:
        return False # Per specification for SPY/V00

    # Same issuer, different funds - requires deeper analysis
    holdings_overlap = calculate_holdings_overlap(sec1, sec2)
    if holdings_overlap > 0.98 and sec1.benchmark_index ==
sec2.benchmark_index:
        # High risk, potentially identical. Default to blocking.
        return True

    return False # Default to not identical if no rule is met

```

...

3.3 Recommended Libraries and Tools

- **Language:** Python
- **Data Analysis:** `pandas` , `numpy`
- **Database:** PostgreSQL (for auditability) or Redis (for performance).
- **Data Sources:** A reliable financial data provider API (e.g., Polygon, Refinitiv, Bloomberg) for security master data and ETF holdings.

4.0 Critical Analysis

4.1 Potential Failure Modes

1. **Stale or Inaccurate Data:** The SIS module is critically dependent on accurate security master and holdings data. If the data provider has a lag, a new ETF might be misclassified, or changes in an index might not be reflected.
 - **Mitigation:** Implement data quality checks and use multiple data sources as a cross-reference. Have a clear "fail-closed" policy where trades are blocked if data is unavailable or untrusted.

2. **Incomplete Cost Basis:** If shares are transferred in from another brokerage without cost basis information, the **Loss Realization Event Trigger** will fail or use an incorrect basis, failing to create a necessary restriction.

- **Mitigation:** The system must enforce that all positions have a valid cost basis before allowing a sale, or flag such sales for manual review.

3. **System Bypass:** A user could potentially place a trade through a different channel (e.g., calling a human broker) that bypasses the pre-trade check.

- **Mitigation:** All order entry points must be integrated with the compliance gateway. Post-trade (T+1) reconciliation reports are essential to catch any violations that slip through.

4.2 Edge Cases

1. **The "30 Days Before" Rule:** The specification focuses on blocking re-entry *after* a sale. The IRS rule is a 61-day window (30 days before, the day of the sale, and 30 days after). A truly compliant system must also scan for purchases made in the 30 days *prior* to the sale at a loss.

- **Enhancement:** When a loss is realized, the `on_position_close` function must also query the trade blotter for any purchases of the same or substantially identical securities within the last 30 days. If found, that purchase is now a wash sale, and its cost basis must be adjusted accordingly.

2. **Options and Derivatives:** Selling a stock at a loss and then buying a call option on that same stock is a wash sale. The SIS module must be aware of the stock/option relationship.

- **Enhancement:** The security master data must include underlying relationships for all derivatives. The `are_substantially_identical` function needs explicit rules for this.

3. **Multiple Accounts:** The wash sale rule applies across all of an individual's accounts, including IRAs.

- **Enhancement:** The `user_id` must be a universal identifier for an individual, not a specific account number. The system must have a holistic view of the user's trading activity.

4. **Corporate Actions:** A stock symbol change, merger, or spin-off could invalidate entries in the **Restricted Securities Ledger**.

- **Enhancement:** The system needs a process to listen for corporate action events and update the ledger accordingly (e.g., if `XYZ` becomes `ABC`, the restriction on `XYZ` should be transferred to `ABC`).

4.3 Optimizations

1. **Caching:** The "substantially identical" status between two securities rarely changes. Results from the SIS module should be aggressively cached (e.g., in Redis) with a long TTL (e.g., 24 hours) to reduce database load and latency in the pre-trade check.

2. **Risk-Based Configuration:** Instead of a binary `ALLOW / REJECT`, the SIS module could output a risk score (e.g., 0.0 to 1.0). The pre-trade check can then use a configurable threshold. This allows the firm to set its own risk tolerance (e.g., "conservative" setting blocks anything with a score > 0.8 , "aggressive" blocks > 0.95). This directly addresses the ambiguity of the IRS rule.